

G03 Review A - PROTECTED

OPEN ONLY AFTER THE MATCHING GATE ATTEMPT HAS BEEN SAVED. This is diagnostic; it is not a copyable solution.

Objective control panel

Control	Expected
Raw entity rows	5
Raw fact rows	11
Duplicate dimension keys	0
Duplicate fact IDs	1
Bad-date IDs	O108
Invalid-amount IDs	O109
Unmatched-FK IDs	O107
Rows blocked by duplicate dimension key	none
Accepted rows	7
Quarantined rows	4
Eligible accepted rows	5
Eligible accepted amount	425.00
API records raw	6
API unique IDs	5
API duplicate IDs	E5

Accepted Completed controls by region

Group	Count	Amount
East	2	165.00
North	1	100.00
South	1	90.00
West	1	70.00

Critical-pattern diagnosis

K1 - Python reasoning and core language control

Control flow/data structures/functions are tangled into one script; bad input is treated as a normal value; exception handling hides the defect; code cannot be explained from state to output.

K2 - Files, pandas, grain, joins and validation

Raw input is overwritten; duplicate IDs/date/amount/FK issues are silently dropped; a non-unique dimension key causes fan-out; final aggregate lacks independent reconciliation.

K3 - APIs, pagination, retries, configuration and secret boundaries

Only page 1 is read; duplicate API IDs are double-counted; retry loops are unbounded; requests has no timeout/status check; secrets/config values are hard-coded or logged.

K4 - Reusable pipeline structure, logging, testing and DB interface discipline

All behavior lives at import time; logging/tests do not cover failure paths; DB interaction cannot be isolated/tested; output success is claimed without validation.

K5 - Linux/WSL shell, permissions, venv and command safety

Working directory/target is uncertain; sudo/chmod 777 is used as a generic repair; exit codes do not distinguish failure; .venv/runtime files are tracked.

K6 - Git/GitHub state, branches, conflicts, remotes, PRs and recovery

Staging/commit state is not understood; conflict is "resolved" by deleting required meaning; fetch/pull are confused; destructive reset rewrites shared history; recovery evidence is missing.

K7 - Integrated reproducibility, evidence and technical explanation

Clean clone depends on hidden files/manual edits; failed attempt was overwritten; evidence does not trace claim -> command/code -> observed result -> validation; explanation cannot defend the controls.

Targeted changed repair menu

Weak	Fresh changed repair proof
K1	Write a new 25-40 line parser/validator for a different tiny 6-row in-memory dataset. Use one function, one collection choice, one deliberate exception and
K2	Use a changed 6-row CSV you create yourself. Preserve raw, quarantine one bad row, prove a many-to-one join key is unique, aggregate at a changed
K3	Use a three-page synthetic JSON payload with a different cursor field. Detect one duplicated event ID; design bounded retry + timeout + config precedence
K4	Refactor one side effect out of import-time code; add two tests including a failure path; add structured log evidence; replace a direct DB call with an injected
K5	In a fresh sandbox, use pwd/ls before file changes, build one pipe+redirection evidence command, create a shell runner with exit 0/2, and recreate an ignored
K6	Create a new repo with staged + unstaged state, a conflict in a different file, a safe revert, reflog evidence, and a local remote/second clone fetch-before
K7	Clone a different fresh repository into an empty directory and rerun it from README only. Record every hidden assumption, repair the repo, clone again

Routing after repair

Append each repair as new evidence. If all weak competencies are now demonstrated and a full reassessment is not justified, G03 may pass without exposing the next sealed Gate. If a fresh full Gate is needed, explicitly assign the next variant. Never erase the original attempt.